

L4-P05 练习手册

GPTQ · AWQ 量化技术

实践 1 执行 GPTQ 量化 (4-bit)

量化程序文件: 1_qwen15_autogptq_quantize.py

1.1 实践目的

学习如何使用 AutoGPTQ 工具，将 FP16 全精度模型量化为 INT4 格式，从而显著降低显存占用并加速模型推理。

1.2 环境准备

在 MoPaaS 平台上创建开发环境，选择 PyTorch 2.4 镜像（已包含常用依赖）。随后安装 AutoGPTQ：

```
pip install auto-gptq
```

1.3 核心原理

GPTQ (Generative Pre-trained Transformer Quantization) 是一种基于二阶信息的逐层后训练量化算法，其核心思路可概括为以下四点：

#	原理要点	详细说明
①	逼近层输出，而非权重本身	GPTQ 不追求每个权重数值精确还原，而是最小化量化前后的层输出差异，即保证 $Wx \approx \hat{W}x$ 。
②	利用校准数据判断权重敏感度	通过一小批真实输入数据，计算 Hessian 信息，判断哪些方向误差影响大、哪些权重更敏感，从而指导量化优先级。

#	原理要点	详细说明
③	逐列（按块）量化	每次只压缩一小部分权重（通常按列或按 block），将该部分量化为 4-bit，控制单次引入的误差量。
④	误差补偿（核心创新）	量化完一列后，将本次产生的量化误差“分摊”到后续尚未量化的权重上，使得局部有误差但整体层输出仍接近原模型。

💡 关键公式

量化目标： $Wx \approx \hat{W}x$

其中 W 为原始 FP16 权重， $\hat{W}$ 为量化后的 INT4 权重， x 为输入激活值。

GPTQ 通过最小化输出误差 $\|Wx - \hat{W}x\|^2$ 来寻找最优量化参数。

1.4 程序执行流程

步骤	操作	说明
1	导入依赖库	载入 transformers 和 auto_gptq，为模型加载与量化做准备。
2	设置量化参数	指定模型路径、输出路径、量化位宽（4-bit）、分组大小（group size）和校准文本。
3	加载分词器	使用 AutoTokenizer 将校准样本文本转换为模型输入 token。
4	创建量化配置	通过 BaseQuantizeConfig 定义量化策略，包括 bit 宽、分组方式、是否量化激活值。
5	加载模型并量化	使用 AutoGPTQForCausalLM.from_pretrained() 加载 FP16 模型，调用 model.quantize() 根据校准数据计算统计量并对权重量化。
6	保存量化结果	调用 model.save_quantized() 将量化权重与配置文件写入指定目录，生成可直接推理的 INT4 模型。
7	输出结果与耗时	打印量化完成提示、耗时及保存路径。

☒ 预期输出

模型量化完成并保存到：../models/Qwen1.5-1.8B-GPTQ-4bit

1.5 量化前后性能比较

性能对比程序文件：2_compare_qwen15_perf.py

该程序自动加载 FP16 与 INT4 两种版本模型，并在相同条件下测量以下指标，生成可视化对比表格：

测量指标	FP16 原始模型	INT4 (GPTQ) 量化模型
加载时间 (s)	较长	通常更短
显存占用 (MB)	较高 (约 3.8 GB)	显著降低 (约 1.2 GB)
推理耗时 (s)	基准	视硬件而定
推理速度 (tok/s)	基准	视硬件而定
模型体积 (GB)	基准	约为 FP16 的 1/3

⚠ 注意事项

部分国产 GPU 暂不支持 4-bit CUDA 算子，INT4 推理速度优势在此类环境中可能无法体现。

性能数据因硬件环境不同而存在差异，以实际运行结果为准。

实践 2 激活感知量化 (AWQ)

量化程序文件: 1_awq.py

2.1 实践目的

对 Qwen1.5-1.8B-Chat 模型执行 AWQ (Activation-aware Weight Quantization) 离线量化, 生成 INT4 低比特模型, 在减小显存占用、加速推理的同时保持精度稳定。

最终输出模型路径: ../models/Qwen1.5-1.8B-Chat-AWQ-INT4

2.2 兼容环境

依赖项	版本要求
Python	≥ 3.10
torch	2.4.x ~ 2.6.x (实测: 2.4.1+das.opt2.dtk2504)
transformers	4.51.x (推荐: 4.51.3)
autoawq	0.2.9
accelerate	1.4.0
tokenizers	0.21.4

在 MoPaaS 平台安装依赖:

```
pip install transformers==4.51.3 autoawq==0.2.9

# 验证安装结果

pip list | grep -E "torch|transformers|autoawq|accelerate|tokenizers"
```

2.3 AWQ 算法原理

AWQ 与 GPTQ 的核心区别在于：AWQ 通过采样真实输入的激活分布来指导权重量化，而非依赖 Hessian 矩阵。

2.3.1 量化公式

AWQ 的量化过程可表示为：

核心公式

$$w_int4 = \text{round}(w_fp16 / s) + z$$

参数说明：

s：缩放因子（scale），由该层激活值的方差计算得出

z：零点（zero-point），用于非对称量化（可选）

量化粒度：按通道（per-channel）独立缩放，保留主要特征响应

2.3.2 量化策略配置

策略项	配置值	说明
量化精度	INT4 (w_bit=4)	每个权重用 4 位整数表示，显存占用降至 FP16 的约 1/4
分组大小	q_group_size ∈ {128, 64}	控制每组共享一套量化参数的权重数量，越小精度越高但开销越大
零点模式	zero_point ∈ {False, True}	False 为对称量化，True 为非对称量化，非对称精度略高
校准样本	离线文本（长句） + token ID 序列	模拟真实推理分布，用于统计激活方差以计算 scale
预热阶段	前向传播一次	确保所有激活统计钩子正常初始化

策略项	配置值	说明
重试机制	自动尝试 8 组配置	依次遍历所有参数组合，任一成功即停止，容错性强
模型保存	save_quantized() 优先	若接口不存在则回退到 save_pretrained(safetensors=True)

2.4 程序核心功能

功能模块	说明
☒ 自动环境检测	启动时打印当前 torch / transformers / autoawq 版本，便于排查兼容性问题
☒ 动态模块导入	自动导入 awq 包（autoawq 安装后实际包名为 awq），适配不同安装方式
☒ 离线校准文本生成	自动构造 7 条中英文混合长文本样本（500~1000 tokens），模拟真实推理分布
☒ 多轮量化重试机制	自动遍历 8 种参数组合（输入格式 × 分组大小 × 零点模式），确保找到稳定配置
☒ 单 GPU 离线运行	支持单卡环境，自动关闭 KV 缓存（use_cache=False）以防显存溢出
☒ 两种校准数据格式	同时支持原始文本字符串输入和 Token ID 数组（整数）输入，兼容性更强
☒ 智能保存接口	自动检测并选择 save_quantized() 或 save_pretrained() 接口
☒ 推理自检	量化完成后通过 pipeline("text-generation") 执行一次生成验证，确认模型可用

2.5 主要执行流程

程序按以下 8 个阶段顺序执行，每个阶段均有详细日志输出：

阶段	操作	详细说明
①	环境检查	打印当前 torch、transformers、autoawq 的版本；确认 awq 模块可正常导入。
②	加载模型与分词器	使用 AutoTokenizer 与 AutoAWQForCausalLM 从 MODEL_IN 加载模型；设置 padding_side="left" 与自动补充 pad_token_id；模型以 FP16 格式载入 GPU，关闭 KV 缓存。
③	构建校准样本	自动生成 7 条中英文长文本（重复拼接形成 500~1000 tokens），打印每条样本 token 数，用于统计激活分布。
④	模型预热 (Prewarm)	依次输入校准样本执行前向传播，触发每层激活统计钩子初始化。
⑤	准备量化输入数据	将长文本转换为 token ID 数组 (to_token_id_batches)，每条样本长度固定为 256~512 tokens。
⑥	多轮量化尝试	按 strings/ids × g128/g64 × zp=False/True 共 8 种组合依次尝试，每次失败自动打印错误并继续下一组。
⑦	保存量化模型	成功后优先调用 save_quantized()；若接口不存在则使用 save_pretrained(safetensors=True)；同步保存 tokenizer。
⑧	推理自检 (可选)	通过 pipeline("text-generation") 生成一句话，验证量化模型可用性；若推理失败则打印"跳过自检"并继续。

2.5.1 多轮量化配置清单

第 ⑥ 阶段自动按如下顺序尝试 8 种量化配置：

#	输入格式	分组大小 (q_group_size)	零点模式 (zero_point)	优先级
1	strings (原始文本字符串)	128	False (对称)	最高
2	strings	64	False	
3	strings	128	True (非对称)	
4	strings	64	True	

#	输入格式	分组大小 (q_group_size)	零点模式 (zero_point)	优先级
5	ids (Token ID 数组)	128	False	
6	ids	64	False	
7	ids	128	True	
8	ids	64	True	最低 (兜底)

附录 GPTQ 与 AWQ 算法横向对比

下表从多个维度对比两种主流后训练量化方法，供选型参考：

对比维度	GPTQ	AWQ
量化依据	基于 Hessian 二阶信息	基于激活值分布 (方差)
量化粒度	逐列/逐块量化 + 误差补偿	按通道 (per-channel) 独立缩放
校准数据	少量文本 (通常 128 条)	少量文本 (7~128 条)
量化精度	INT4 (可选 INT3/INT8)	INT4
速度 (量化过程)	较慢 (需要计算 Hessian)	较快
精度损失	极低 (误差补偿机制)	极低 (激活感知缩放)
硬件兼容性	部分国产 GPU 不支持 4-bit 算子	部分国产 GPU 不支持 4-bit 算子
工具库	auto-gptq	autoawq (包名 awq)
适用场景	对精度要求极高的生产部署	快速验证 + 灵活配置实验